



# HTML+CSS

---



# Table

To make a cell span over multiple rows, use the **rowspan** attribute

To make a cell span over multiple columns, use the **colspan** attribute:

| NAME |  |  |
|------|--|--|
|      |  |  |
|      |  |  |
|      |  |  |
|      |  |  |

| APRIL |  |  |
|-------|--|--|
|       |  |  |
|       |  |  |
|       |  |  |
|       |  |  |

# Solution

```
<table>
```

```
  <tr>
```

```
    <td colspan=2>Name</td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
</table>
```

```
<table>
```

```
  <tr>
```

```
    <td rowspan=3>April</td>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td></td>
```

```
    <td></td>
```

```
    <td></td>
```

```
  </tr>
```

```
</table>
```





# CSS - Cascading Style Sheet

---

<https://www.w3schools.com/cssref/index.php>

# Inline Styles

Style rules placed within an HTML element via the style attribute

```
<h1>Share Your Travels</h1>
<h2>style="font-size: 24pt"Description</h2>
...
<h2>style="font-size: 24pt; font-weight: bold;">Reviews</h2>
```

**LISTING 3.1** Internal styles example

An inline style only affects the element it is defined within and will override any other style definitions for the properties used in the inline style.

Using inline styles is generally discouraged since they increase bandwidth and decrease maintainability.

Inline styles can however be handy for quickly testing out a style change.

# Embedded Style Sheet

Style rules placed within the `<style>` element inside the `<head>` element

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <style>
    h1 { font-size: 24pt; }
    h2 {
      font-size: 18pt;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Share Your Travels</h1>
  <h2>New York - Central Park</h2>
  ...
</body>
```

**LISTING 3.2** Embedded styles example

While better than inline styles, using embedded styles is also by and large discouraged.

Since each HTML document has its own `<style>` element, it is more difficult to consistently style multiple documents when using embedded styles.

# External Style Sheet

Style rules placed within an external text file with the .css extension

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <link rel="stylesheet" href="styles.css" />
</head>
```

**LISTING 3.3** Referencing an external style sheet

This is by far the most common place to locate style rules because it provides the best maintainability.

- When you make a change to an external style sheet, all HTML documents that reference that style sheet will automatically use the updated version.
- The browser is able to cache the external style sheet which can improve the performance of the site

# CSS Selector

- A CSS selector is a pattern used to select and style specific elements on a web page. It defines which HTML elements the CSS rules will apply to.

- Types of CSS Selectors:

- **Universal Selector (\*)**: Matches all elements.

```
* {  
    margin: 0;  
    padding: 0;  
}
```

- **Type/Element Selector**: Matches elements by their tag name.

```
p {  
    color: red;  
    text-align: center;  
}
```





# CSS Selector

- **Class Selector (\*)**: Matches elements with a specific class.

```
.button-container {  
    background-color: green;  
}
```

- **ID Selector (#id)**: Matches an element with a specific ID (unique per page).

```
#header {  
    font-size: 24px;  
}
```

- **Attribute Selector**: Matches elements based on attributes and their values.

```
input[type="text"] {  
    border: 1px solid black;  
}
```

- **Group Selector**: Groups multiple selectors to apply the same styles.

```
h1, h2, h3 {  
    font-family: Arial, sans-serif;  
}
```



# CSS Selector

- **Descendant Selector:** Matches elements nested within a specific parent.

```
div p {  
    color: gray;  
}
```

- **Child Selector (>):** Matches direct child elements.

```
ul > li {  
    list-style: none;  
}
```

- **Pseudo-classes:** Matches elements based on a state or position.

```
a:hover {  
    text-decoration: underline;  
}
```



# CSS Selector

HTML elements can also refer to more than one class.

```
<p class="center large">This paragraph refers  
to two classes.</p>
```



# Properties and Values - Text

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>
```

Hello World

```
<p style="color:#FFFFFF;">Lorem ipsum...</p>
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

```
<h1 style="border:2px solid Tomato;">Hello World</h1>
```

Hello World

# Properties and Values

```
body {  
  background-image: url("paper.gif");  
}
```

**Hello World!**

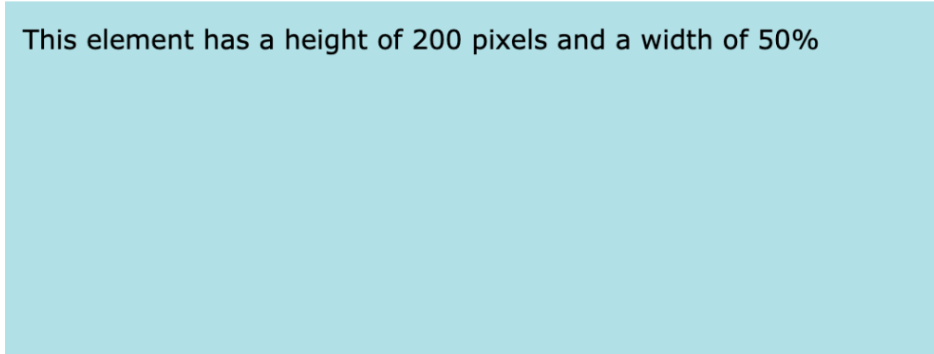
This page has an image as the background!



# Height x Width

## CSS height and width Examples

This element has a height of 200 pixels and a width of 50%



### Example

Set the height and width of a <div> element:

```
div {  
  height: 200px;  
  width: 50%;  
  background-color: powderblue;  
}
```



# Font

1. font-size: 40px; / 2.5em; / 100%;
2. font-family: Arial, "Times New Roman"
3. font-style: normal/italics/bold;
4. font-weight: normal/bold/900;



# CSS Borders

```
p.dotted {border-style: dotted;}
```

```
p.dashed {border-style: dashed;}
```

```
p.solid {border-style: solid;}
```

```
p.double {border-style: double;}
```

```
p.groove {border-style: groove;}
```

```
p.ridge {border-style: ridge;}
```

```
p.inset {border-style: inset;}
```

```
p.outset {border-style: outset;}
```

```
p.none {border-style: none;}
```

```
p.hidden {border-style: hidden;}
```

```
p.mix {border-style: dotted  
dashed solid double;}
```

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border. The effect depends on the border-color value.

A ridge border. The effect depends on the border-color value.

An inset border. The effect depends on the border-color value.

An outset border. The effect depends on the border-color value.

No border.

A hidden border.

A mixed border.



# CSS Borders

- ▶ `border-width: 5px;`
- ▶ `border-color: red;`

5px border-width

medium border-width

- ▶ `border-bottom: 1px solid #ccc;`
- ▶ `border-top: 1px solid #ccc !important;`
- ▶ `border-right: 1px solid #ccc;`
- ▶ `border-left: 1px solid #ccc !important;`



# Align

float: left (default in english)/right

text-align: center/left/right

Different: "text-align" applies to the *content* of a box, while "float" applies to the box itself.



# Margins and Paddings

Margins are used to create space around elements, outside of any defined borders.

```
margin: 25px;
```

```
margin-top: 10px;
```

```
margin: 25px 50px 75px 100px;
```

- top margin is 25px - right margin is 50px - bottom margin is 75px - left margin is 100px

This element has a margin of 70px.

Padding is used to create space around an element's content, inside of any defined borders.

```
padding: 25px;
```

```
padding-top: 10px;
```

```
padding: 25px 50px 75px 100px;
```

- top padding is 25px - right padding is 50px - bottom padding is 75px - left padding is 100px

This element has a padding of 70px.

# Anchor style

- **a:link** - a normal, unvisited link
- **a:visited** - a link the user has visited
- **a:hover** - a link when the user mouses
- **a:active** - a link the moment it is click

```
a:link {  
  background-color: white;  
  color: black;  
  border: 2px solid green;  
  padding: 10px 20px;  
  text-align: center;  
  text-decoration: none;  
}
```



This is a link

# Specificity

Specificity measure which css rule to apply when multiple css rules target the same styling property of an element, i.e.,

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p {color: red;}

      #p_id {color: black;}

      .class{
        color: yellow;
      }
    </style>
  </head>
  <body>
    <p style="color: green" id="p_id" class="class">Hello World!</p>
  </body>
</html>
```



# Specificity

Browsers measure specificity using the following scoring:

1. html style attribute (1000 points)
2. Id (100 points)
3. Class, attribute selector, pseudo class (10 points)
4. element selector (1 point)
5. Universal selector (0 point)



# Specificity Examples

**#id:** Specificity Score=100

**.class:** Specificity Score=10

**#id.class p:** Specificity score =  $100 + 10 + 1 = 111$

**.class p: hover:** Specificity score =  $10 + 10 + 1 = 21$

**div:** Specificity score = 1



# CSS Cascade

If two CSS rules have the same specificity, the browser determines which rule to apply using the following criteria:

**Order of Appearance:** For internal/external css, the rule that is defined later in code has more precedence.





# Class Test

## HTML & CSS Test

### Part 1: HTML

Create an HTML page that includes the following elements:

- A big header text with a title (e.g., "My Simple Web Page").
- A main section with at least one paragraph and an image.
- Another section that includes your name and the current year.

### Part 2: CSS

Write a CSS stylesheet that styles your HTML page. Include the following styles:

- Use a font size and color for the header.
- Change the background color of the page.
- Style the main content section, giving the paragraph some padding and margin.
- Style the second section with a different background color.

# Media Queries

```
body {  
    font-size: 18px;  
    background-color: red;  
}  
  
/* For screens smaller than 768px */  
@media (max-width: 768px) {  
    body {  
        font-size: 16px;  
    }  
}  
  
/* For screens smaller than 600px */  
@media (max-width: 600px) {  
    body {  
        background-color: lightblue;  
    }  
}  
  
/* For screens smaller than 480px */  
@media (max-width: 480px) {  
    body {  
        font-size: 14px;  
    }  
}
```

# Media Queries

```
/* Default image size inside .responsive-img class */
.responsive-img {
  width: 400px;
  height: 300px;
}

/* For small screens (max-width: 600px) */
@media (max-width: 600px) {
  .responsive-img {
    width: 100%; /* Make the image fill 100% of the container width */
    height: auto; /* Adjust height automatically to maintain aspect ratio */
  }
}

/* For larger screens (min-width: 1024px) */
@media (min-width: 1024px) {
  .responsive-img {
    width: 600px; /* Larger image for bigger screens */
    height: auto; /* Maintain aspect ratio */
  }
}
```



# CSS Position Property

In CSS, static, absolute, and relative and fixed are values for the position property, and they determine how an element is positioned in the layout. Here's an explanation of each:

## static (Default Positioning):

1. All elements are static by default unless explicitly specified otherwise.
2. The element is positioned according to the normal flow of the document.
3. Top, right, bottom, and left properties have no effect.

## relative (Positioned Relative to Its Normal Position):

1. The element remains in the normal document flow.
2. Its position is adjusted relative to its original position using top, right, bottom, or left.
3. It does not affect the position of other elements.
4. When an element has position: relative, its original position in the normal document flow is reserved, even though the element can be visually offset using the top, right, bottom, or left properties.
5. **Usage:** can be used to make an ancestor positioned so that an absolutely positioned descendant can be moved using top, left, right, and bottom with respect to that ancestor. Can also be used to nudge an element on hover (however, using **transform: translateY(-5px)** is a better approach for nudging).

```
.element {  
  position: relative;  
  top: 0;  
  transition: top 0.2s ease-in-out;  
}  
  
.element:hover {  
  top: -5px; /* Nudges up by 5 pixels */  
}
```



# CSS Position Property

In CSS, static, absolute, and relative and fixed are values for the position property, and they determine how an element is positioned in the layout. Here's an explanation of each:

## **absolute (Positioned Relative to the Nearest Positioned Ancestor):**

1. The element is removed from the normal document flow, i.e., other elements behave as if this element does not exist.
2. It is positioned relative to the nearest ancestor with a position value other than static (like relative, absolute, or fixed). If no such ancestor exists, it is positioned relative to the <body> element.
3. If no positioning offsets (top, right, bottom, left) are provided, the element stays in its natural position as if it were still in the normal document flow. However, it will overlap with other elements, because for other elements, it does not exist in the document flow
4. **Usage:** can be used to display popup or tool-tip over a certain element/container.

## **fixed (Positioned Relative to View Port):**

1. The element is removed from the normal document flow, i.e., other elements behave as if this element does not exist.
2. The fixed position in CSS is used to place an element relative to the viewport.
3. An element with position: fixed always stays in the same position on the screen, even when the user scrolls.
4. If no positioning offsets (top, right, bottom, left) are provided, the element stays in its natural position as if it were still in the normal document flow. However, it will overlap with other elements, because for other elements, it does not exist in the document flow.
5. **Usage:** can be used to display top navigation header, display popup over viewport, etc.



# CSS Position Property

In CSS, static, absolute, and relative and fixed are values for the position property, and they determine how an element is positioned in the layout. Here's an explanation of each:

## **Sticky (hybrid of relative and fixed):**

1. The position: sticky; CSS property allows an element to "stick" to a specific position within its containing block when the user scrolls past it. This is a hybrid of relative and fixed positioning.
2. Initially, the element behaves like a relative element, meaning it is positioned normally in the document flow.
3. When the user scrolls and the element reaches a specified offset (using top, bottom, left, or right), it becomes "stuck" to that point.



# CSS Position Property

**Examples:**

[https://www.w3schools.com/css/css\\_positioning.asp](https://www.w3schools.com/css/css_positioning.asp)